

경력기술서

안병찬 | Unity Client Developer

지원 직무	핵심 경험	주요 기술
Unity 개발자	2D 디펜스, 3D 액션 RPG, Android 빌드 테스트	Unity, C#, DOTS/ECS, Addressables

핵심 요약

구분	내용
Unity 클라이언트 개발	2D 디펜스 게임과 3D 액션 RPG 프로젝트에서 플레이어 조작, 전투, UI, 씬 전환, 저장 구조를 구현했습니다.
ECS 구조 경험	Slime Defense 프로젝트에서 DOTS/ECS 기반 적 스폰 시스템과 웨이포인트 기반 이동 흐름을 구현했습니다.
빌드 및 검증 경험	Addressables, SubScene, Android APK 빌드, ADB 설치 테스트 과정에서 발생한 문제를 확인하고 해결했습니다.
문제 해결 방식	현상 확인, 재현 조건 확인, 원인 범위 축소, 수정, 재검증 순서로 문제를 해결했습니다.

1. Slime Defense

프로젝트 개요

Unity 로 개발한 2D 디펜스 게임 프로젝트입니다. 플레이어가 슬라임을 성장시키고, 몰려오는 적을 처치하며 스테이지를 진행하는 구조로 설계했습니다.

단순한 전투 구현에 그치지 않고, 적 스폰, 투사체 충돌, 성장 요소, 씬 전환, Addressables 기반 리소스 관리, Android 빌드 테스트까지 포함하여 구현과 검증을 진행했습니다.

담당 역할

프로젝트 전반의 구조 설계와 Unity 클라이언트 구현을 담당했습니다. 게임 진행 흐름, 적 스폰, 투사체 처리, UI 연동, 데이터 관리, 씬 로딩, Android 빌드 테스트를 직접 구현하고 검증했습니다.

주요 구현 내용

게임 흐름 및 매니저 구조 구현

게임의 시작, 진행, 종료 흐름을 관리하기 위해 매니저 구조를 설계했습니다. 로딩 씬, 인게임 씬, UI 표시 상태를 분리하여 관리했고, 각 기능이 한 곳에 과하게 의존하지 않도록 역할을 나누었습니다.

로딩 화면에서는 배경 이미지, 로딩 바, 시작 안내 UI 를 관리하고, 최소 로딩 시간을 적용하여 씬 전환이 갑작스럽게 보이지 않도록 처리했습니다.

DOTS/ECS 기반 적 스폰 시스템 구현

적이 일정 조건에 따라 생성되고 이동할 수 있도록 ECS 기반 스폰 시스템을 구현했습니다. 웨이포인트 버퍼를 기준으로 적의 이동 경로를 설정하고, 스폰 데이터에 따라 적이 순차적으로 생성되도록 구성했습니다.

스폰 시스템에서는 불필요한 객체 생성과 반복 연산을 줄이는 방향으로 구조를 잡았고, EntityCommandBuffer 를 활용해 엔티티 생성과 컴포넌트 추가 흐름을 관리했습니다.

투사체 충돌 및 판정 구조 개선

전투 중 투사체가 의도한 대상 수보다 많은 적을 관통하는 문제를 확인했습니다. 처음에는 대미지 부족, 적 체력 설정, 엔티티 버퍼 용량 문제 등을 의심했지만, 반복 테스트를 통해 충돌 판정 기준과 렌더링 기준이 섞여 있는 구조가 원인임을 확인했습니다.

이후 충돌 판정 기준과 시각적 렌더링 기준을 분리하여 문제를 수정했습니다. 수정 후에는 같은 조건에서 다시 플레이하며 적 처치, 투사체 충돌, 관통 횟수가 의도대로 동작하는지 재검증했습니다.

Addressables 및 SubScene 빌드 문제 해결

Addressables 를 사용하는 구조에서 빌드 후 SubScene 이 정상적으로 로드되지 않는 문제를 확인했습니다. 처음에는 코드상 로딩 문제를 의심했지만, 빌드에 필요한 SubScene 참조가 포함되지 않는 구조가 원인일 가능성을 확인했습니다.

이후 부트 씬에서 SubScene 참조를 명확히 잡는 방식으로 수정하여 빌드 시 SubScene 이 누락되지 않도록 처리했습니다. 수정 후 실제 Android 빌드 환경에서 다시 실행하며 씬 로딩 여부를 확인했습니다.

Android APK 빌드 및 실제 기기 테스트

Unity Android 빌드를 진행하고, APK 파일을 실제 기기에 설치하여 테스트했습니다. ADB 를 활용해 기기 연결 상태를 확인하고, 빌드 설치와 실행 과정을 검증했습니다.

빌드 과정에서 JDK 경로 문제, APK 파일 잠금 문제, 실행 파일 공유 방식 등을 확인하며 실제 배포 과정에서 발생할 수 있는 문제를 경험했습니다.

사용 기술

Unity, C#, DOTS/ECS, Burst, EntityCommandBuffer, Addressables, SubScene, Android Build, ADB, Git, GitHub

2. Monster World

프로젝트 개요

Unity 로 개발한 3D 액션 RPG 프로젝트입니다. 플레이어가 몬스터와 전투하고, 경험치를 획득해 성장하며, 몬스터를 포획하고 소환할 수 있는 구조로 설계했습니다.

전투, 성장, 몬스터 AI, 저장 시스템, UI 관리 기능을 구현하며 3D 게임에서 필요한 기본적인 플레이 흐름을 직접 구성했습니다.

담당 역할

3D 액션 RPG 의 플레이어 조작, 전투 시스템, 몬스터 AI, 성장 시스템, 포획 및 소환 기능, 저장 구조 구현을 담당했습니다. 플레이어와 몬스터, UI, 스킬, 투사체, 이펙트 시스템이 함께 동작할 수 있도록 게임 흐름을 관리했습니다.

주요 구현 내용

플레이어 조작 및 전투 기능 구현

플레이어 이동, 조준, 무기 전환, 원거리 공격 기능을 구현했습니다. 3D 공간에서 카메라 방향과 캐릭터 방향이 어긋나는 문제를 해결하기 위해 조준 방향과 캐릭터 회전 방향을 맞추는 구조를 설계했습니다.

이를 통해 플레이어가 바라보는 방향과 공격 방향이 자연스럽게 연결되도록 만들고, 전투 중 조작감이 어색하지 않도록 개선했습니다.

몬스터 AI 구현

몬스터가 순찰, 추적, 공격, 복귀 상태를 오갈 수 있도록 상태머신 기반 AI 를 구현했습니다. 플레이어와의 거리, 감지 범위, 공격 가능 여부에 따라 상태가 전환되도록 구성했습니다.

몬스터가 단순히 플레이어를 따라오는 방식이 아니라, 상황에 따라 행동을 바꾸도록 설계하여 전투 흐름이 자연스럽게 이어지도록 했습니다.

성장 및 경험치 시스템 구현

몬스터 처치 후 경험치를 획득하고, 일정 경험치에 도달하면 레벨업이 이루어지는 성장 시스템을 구현했습니다. 레벨업에 따라 플레이어 능력치가 변화하도록 구성하여 전투 결과가 성장으로 이어지는 구조를 만들었습니다.

몬스터 포획 및 소환 시스템 구현

전투 중 몬스터를 포획하고, 저장된 몬스터를 다시 소환할 수 있는 기능을 구현했습니다. 단순 처치 중심의 전투가 아니라 수집과 운용 요소를 포함할 수 있도록 구조를 설계했습니다.

저장 시스템 구현

슬롯 기반 저장 시스템을 구현하여 플레이 데이터를 유지할 수 있도록 했습니다. 플레이어 정보, 성장 상태, 몬스터 관련 데이터를 저장하고 다시 불러오는 흐름을 구성했습니다.

사용 기술

Unity, C#, 3D Character Control, State Machine AI, UI Management, Save System, Projectile System, Camera Direction Control

3. 게임 테스트 및 피드백 경험

슈퍼피플 알파/베타 테스트 및 서포터 경험

슈퍼피플 알파/베타 테스트에 참여하며 게임 플레이 중 느낀 문제점과 개선 의견을 정리해 피드백을 제출했습니다. 단순히 재미 여부를 적는 것이 아니라, 플레이 경험을 방해하는 요소를 찾아 원인과 개선 방향을 함께 정리하려고 했습니다.

특히 아이템 파밍 구조, 클래스 간 전투력 차이처럼 게임의 반복 플레이 경험에 영향을 주는 부분을 중심으로 문제를 인식했습니다. 이후 피드백이 인정되어 서포터로 지정된 경험이 있습니다.

이 경험을 통해 게임을 플레이어 입장에서만 보는 것이 아니라, 구조와 밸런스, 반복 플레이 경험의 관점에서 바라보는 습관을 기를 수 있었습니다.

배틀그라운드 비공개 테스트 아르바이트 경험

배틀그라운드 비공개 테스트 아르바이트에 참여하며 실제 테스트 환경에서 이슈를 기록한 경험이 있습니다. 비밀 유지 계약으로 인해 세부 테스트 내용은 공개할 수 없지만, 플레이 중 발생한 문제를 나중에 기억에만 의존하지 않기 위해 상황, 행동, 결과를 구분해 메모했습니다.

문제가 발생한 위치, 직전에 수행한 행동, 주변 조건, 실제 결과를 나누어 기록하며 재현 가능성을 높이는 방식으로 접근했습니다. 이를 통해 버그나 이상 현상을 발견했을 때 개발자와 QA가 확인할 수 있는 형태로 정리하는 중요성을 배웠습니다.

4. 개발자로서의 강점

기능을 구현한 뒤 실제 플레이와 빌드 환경에서 다시 확인하는 과정을 중요하게 생각합니다. Unity 프로젝트를 진행하면서 에디터 안에서만 동작하는 기능이 실제 Android 빌드나 Addressables 환경에서는 다르게 동작할 수 있다는 점을 경험했습니다.

그래서 기능 구현 후에는 직접 실행하고, 문제가 생기면 원인을 추측으로 단정하지 않고 조건을 나누어 확인하려고 합니다. 투사체 충돌 문제, SubScene 로딩 문제, APK 빌드 문제를 해결하는 과정에서도 현상 확인, 원인 범위 축소, 수정, 재검증 순서로 접근했습니다.

또한 게임 테스트 경험을 통해 플레이어가 실제로 불편함을 느끼는 지점을 관찰하는 습관을 길렀습니다. 이 경험은 Unity 개발자로서 단순히 기능을 만드는 것뿐 아니라, 구현된 기능이 실제 플레이 경험에서 자연스럽게 동작하는지 확인하는 데 도움이 되었습니다.

5. 주요 기술 역량

구분	내용
Unity 클라이언트 개발	2D 디펜스 게임과 3D 액션 RPG 프로젝트를 개발하며 플레이어 조작, 전투, 적 스폰, 몬스터 AI, UI, 씬 전환, 저장 시스템을 구현했습니다.
C# 기반 구조 설계	게임 흐름과 기능별 책임을 나누어 구현했습니다. 매니저 구조, 상태머신, 데이터 관리, 이벤트 흐름을 구성했습니다.
DOTS/ECS 활용 경험	엔티티, 컴포넌트, 버퍼, EntityCommandBuffer 를 활용해 다수의 적을 처리하는 구조를 설계했습니다.
Addressables 및 빌드 관리	Addressables 를 활용해 씬과 리소스를 관리하고, Android 빌드 과정에서 발생하는 문제를 확인하고 해결했습니다.
테스트 및 문제 재현	게임 테스트 아르바이트와 개인 프로젝트 디버깅 경험을 통해 발생 조건, 실행 행동, 실제 결과를 구분해 문제를 확인했습니다.